



Learning Module & AI Roadmap Chatbot

Software architecture of the student learning experience — the conversational AI assistant that delivers customized study roadmaps, plus course delivery, assessments, proctoring, transcription and certification — extracted from the Learnify platform.

React 18 + TypeScript + Vite

Supabase (Postgres · Auth · Edge Functions)

Gemini 2.5 Flash via Lovable AI Gateway

Groq Whisper + Llama 3.3

Scope: Learning module & AI chatbot only · **Repository:** shashankraghuram1509/learnify-spotify · **Prepared for:** marketing@prachalas.com

Document type: Technical architecture reference · **Date:** 27 July 2026

01 Overview & Scope

Learnify — Learning Module & AI Roadmap Chatbot · Architecture

Generated 2026-07-27 · from source at commit b88c27f

Learnify is a single-page web application (a Vite + React + TypeScript client) backed entirely by **Supabase** — PostgreSQL with Row-Level Security, JWT authentication, file storage, realtime subscriptions, and Deno-based Edge Functions. This document isolates and describes the **Learning Module**: everything a student touches while learning, and the **AI chatbot** that guides them with customized roadmaps and study help.

IN SCOPE — THE LEARNING MODULE

- **AI Roadmap Chatbot** — floating assistant giving personalized study roadmaps, course help & academic guidance
- **Course delivery** — catalog, enrollment, video player, downloadable materials
- **Assessments** — assignments & timed tests with browser-based proctoring
- **Progress & certification** — progress tracking, completion, certificates
- **AI transcription** — speech-to-text + summary of session recordings

OUT OF SCOPE (REFERENCED ONLY WHERE IT GATES LEARNING)

- Payments / Razorpay — appears only as the premium-enrollment gate
- Placement & jobs (Adzuna, JSearch)
- User-to-user chat & video-call scheduling
- Admin platform analytics, MOU & partner management

02 Technology Stack

FRONTEND (CLIENT SPA)

React 18.3 · TypeScript · Vite 5 · React Router 6 · TanStack Query 5
shadcn/ui · Radix · Tailwind CSS 3 · Framer Motion · Zod · Sonner
jsPDF · html2canvas

BACKEND — SUPABASE

PostgreSQL + RLS · Supabase Auth (JWT) · Storage · Realtime
Edge Functions (Deno)

AI / ML SERVICES

Lovable AI Gateway · google/gemini-2.5-flash · Groq · whisper-large-v3-turbo
Groq · llama-3.3-70b-versatile

THIRD-PARTY INTEGRATIONS

YouTube embed · Google Drive / Calendar / Meet · Razorpay · Brevo email

03 High-Level System Architecture

Four layers. The browser talks only to Supabase; every privileged action and every outbound AI/third-party call is brokered by an Edge Function that first verifies the caller's JWT.

CLIENT · REACT SPA · BROWSER

AIAssistant

floating roadmap chatbot (global)

Course catalog

Courses · CourseDetail

CourseViewer

video · materials · review

Assignments / TakeTest

+ useProctoring hook

Dashboard widgets

progress · certificates

Auth via useAuth · context · data via @supabase/supabase-js (RLS-scoped) · Realtime channels

▼ HTTPS · JWT bearer token ▼

EDGE · SUPABASE EDGE FUNCTIONS (DENO) — AUTH-GATED BROKERS

ai-chat

chatbot → LLM (SSE stream)

create-enrollment

subscription / payment gate

transcribe-video

Whisper + Llama → transcript

upload-to-drive

teacher material upload

create-meet-link

live session (Google Meet)

admin-create-instructor

role provisioning

▼ service-role / RLS ▼ ▼ API keys ▼

DATA · SUPABASE POSTGRES + STORAGE

Learning tables

courses · enrollments · assignments · certificates ...

Identity

profiles · user_roles

Storage

materials · thumbnails

EXTERNAL · AI & 3RD-PARTY

Lovable AI Gateway

Gemini 2.5 Flash

Groq

Whisper · Llama 3.3

Google · YouTube

Drive/Meet · video

04 AI Roadmap Chatbot — Deep Dive

Learnily — Learning Module & AI Roadmap Chatbot · Architecture

Generated 2026-07-27 · from source at commit b88c27f

The headline feature: a conversational assistant that produces customized learning roadmaps, study plans and course help. It is a thin, secure client that streams tokens from a large language model through an auth-gated Edge Function.

CLIENT — AIISSISTANT.TSX

- Global floating widget (bottom-right), mounted once in `App.tsx` across all routes
- Requires an authenticated Supabase session before sending
- `POST` to `${VITE_SUPABASE_URL}/functions/v1/ai-chat` with `Authorization: Bearer <access_token>`
- Reads a **Server-Sent Events** stream, parsing `choices[0].delta.content` and terminating on `[DONE]`
- Renders the reply **incrementally** (token-by-token); handles `429` rate-limit & `402` usage-limit

EDGE FUNCTION — SUPABASE/FUNCTIONS/AI-CHAT

- Decodes the JWT, extracts `sub` (user id); rejects unauthenticated calls with `401`
- Validates `messages[]` : 1–50 messages, each ≤ 4000 chars, role $\in$ {user, assistant, system}
- Prepends the educational **system prompt** (see below) that shapes the roadmap/guidance behaviour
- Calls the LLM with `stream: true` and **proxies the SSE stream straight back** to the browser
- Secret `LOVABLE_API_KEY` never leaves the server

Request & streaming data flow

- 1 **User** opens the widget and sends a message. The client appends it to local `messages` state.
- 2 **AIAssistant** retrieves the Supabase session and POSTs the full message history to the `ai-chat` function with the bearer token.
- 3 **ai-chat** authenticates (JWT `sub`) and validates the payload shape and limits.
- 4 **ai-chat** injects the system prompt and forwards `[system, ...messages]` to **Lovable AI Gateway** → `google/gemini-2.5-flash`, `stream:true`.
- 5 **Gateway/model** streams completion chunks (SSE) back to the function, which returns them as `text/event-stream`.
- 6 **AIAssistant** decodes each `data:` chunk and appends `delta.content` to the on-screen answer until `[DONE]`.

System prompt (verbatim, in `ai-chat/index.ts`):

“You are a helpful educational AI assistant for a learning management system. You help students with their courses, answer questions about assignments, provide study tips, and assist with general academic guidance. Be encouraging, clear, and supportive.”

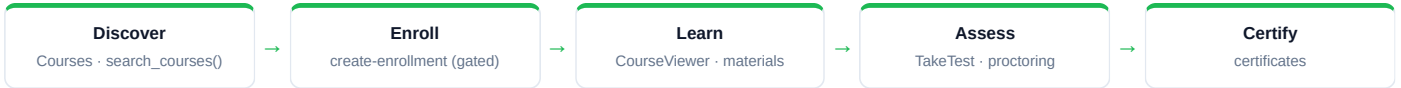
Design note — where the “customized roadmap” comes from. Roadmaps and study plans are produced **dynamically by the LLM** from the running conversation and the system prompt — they are generated per request and are **not persisted** to a dedicated roadmap table. The model receives the message history verbatim; there is currently no server-side injection of the student's enrolled-course or progress data into the prompt (a natural extension point — see §10).

05 The Learning Journey

Learnity — Learning Module & AI Roadmap Chatbot · Architecture

Generated 2026-07-27 · from source at commit b88c27f

End-to-end student path through the module. The AI chatbot overlays every step, always available for guidance.



Discover & Enroll

`Courses` & `CourseDetail` browse the `courses` table (with `modules`). Enrolling calls the `create-enrollment` Edge Function, which enforces access rules server-side:

- **Free courses** → enroll immediately
- **Premium courses** → allowed if `profiles.subscription_tier` ∈ {`Lite`, `Premium`, `Premium Pro`} and unexpired, or a completed `payments` row exists for the course
- Insert into `enrollments` is done with the **service role**, bypassing RLS after checks pass

Learn — `CourseViewer.tsx`

- Verifies enrollment & (for premium) payment/subscription before revealing content
- Embeds the lesson video via a normalized **YouTube** embed URL
- **MaterialsList** lists `course_resources` (pdf/video/link) ordered by `position`
- Shows enrollment `progress` and a **CourseReview**
- Optional **TranscriptViewer** renders AI transcript + summary from `transcripts`

Assess — `AssignmentsList` · `TakeTest`

- Assignments & “tests” share the `assignments` table (distinguished by `type`)
- Starting a test inserts an `assignment_submissions` row (status *pending*) and starts a countdown from `duration_minutes`
- Supports free-text answers or an embedded **Google Form** (`attachment_url`)
- Auto-submits at time-up; results tracked in `student_test_attempts`
- Progress can be boosted via `enrollments.test_progress_bonus`

Track & Certify

- **ProgressTracker** / **EnrolledCoursesList** compute % from `progress` + `test_progress_bonus` (capped 100)
- `video_watch_tracking` records minutes watched & last position
- **CertificateManager** surfaces the student's `certificates` (joined to course titles)

06 Assessment & Proctoring Subsystem

Timed tests are monitored client-side by the `useProctoring` hook, with every event persisted for teacher review.

MONITORED EVENTS → `PROCTORING_LOGS`

`tab switch (visibilitychange)` `window blur` `copy` `paste`
`right-click (contextmenu)` `fullscreen exit`

Fullscreen is requested on start; each violation increments a live counter shown to the student.

ENFORCEMENT → `PROCTORING_VIOLATIONS`

- Aggregates `violation_count` and `failed_attempts` per (assignment, student)
- `blocked_until` can lock a student out after repeated violations
- Enabled per-assignment via `assignments.proctoring_enabled`

07 Data Model

Learnily — Learning Module & AI Roadmap Chatbot · Architecture

Generated 2026-07-27 · from source at commit b88c27f

courses is the hub of the learning schema. Roles live in a separate **user_roles** table (not on **profiles**); subscription state lives on **profiles**. Legend: **PK** primary key · **FK** foreign key.

courses · hub id · title · description teacher_id · thumbnail_url video_url · price · is_premium	enrollments id · student_id course_id → courses progress · test_progress_bonus video_minutes_watched · completed_at	profiles · subscription id (= auth user) full_name · email · college subscription_tier subscription_expires_at
modules id · course_id → courses title · position	lessons id · module_id → modules title · content_type/url/text duration_minutes · position	user_roles · RBAC id · user_id role app_role {student · teacher · admin}
course_resources id · course_id → courses title · resource_type · url file_path · position	course_materials · AI study id · course_id → courses overview · key_points code_examples · practice_exercises	reviews id · course_id · student_id rating · comment
assignments · tests id · course_id → courses type · total_marks · due_date duration_minutes · attachment_url proctoring_enabled	assignment_submissions id · assignment_id → assignments student_id · submission_text/url status · marks_obtained · feedback	student_test_attempts id · assignment_id submission_id → submissions attempt_number · marks · passed
proctoring_logs id · submission_id student_id · event_type event_data · timestamp	proctoring_violations id · assignment_id student_id · violation_count failed_attempts · blocked_until	certificates id · student_id course_id · enrollment_id certificate_url · issued_at
transcripts · AI id · enrollment_id video_url · transcript summary	video_watch_tracking id · student_id · course_id enrollment_id · minutes_watched last_position	taxonomy categories · course_categories tags · course_tags (many-to-many joins)

Key relationships. courses 1—∞ enrollments, modules, course_resources, course_materials, assignments, certificates. modules 1—∞ lessons. assignments 1—∞ assignment_submissions 1—∞ student_test_attempts / proctoring_logs. enrollments 1—∞ transcripts & video_watch_tracking, 1—1 certificates. Identity: profiles holds subscription state; user_roles holds roles, resolved by the DB function has_role(user, role).

08 Edge Functions (learning-relevant)

Deno functions on Supabase. Each verifies the caller before touching data or calling an external API; secrets stay server-side.

FUNCTION	RESPONSIBILITY	EXTERNAL CALL · MODEL
ai-chat	Auth-gate + validate chat; stream LLM completion back to the widget as SSE.	Lovable AI Gateway google/gemini-2.5-flash
transcribe-video	Fetch a recording → transcribe → summarize → insert into <code>transcripts</code> (service role).	Groq whisper-large-v3-turbo + llama-3.3-70b-versatile
create-enrollment	Enroll user after checking subscription tier/expiry or a completed payment; insert <code>enrollments</code> .	Supabase only (service role)
upload-to-drive	Teacher-only; upload a file, then record a <code>course_resources</code> row pointing at it.	Google Drive API (OAuth)
create-meet-link	Create a Google Calendar event with a Meet link for a scheduled live session.	Google Calendar API (OAuth)
admin-create-instructor	Admin-only; provision a teacher auth user, set <code>teacher</code> role, fill profile.	Supabase Admin API
razorpay-* · fetch-*-jobs · send-email	Supporting (payments gate, placement, transactional email) — outside core learning scope.	Razorpay · Adzuna/JSearch · Brevo

09 Authentication & Access Control

AUTH — USEAUTH.TSX

- React context wrapping Supabase Auth; subscribes to `onAuthStateChange` + `getSession()`
- Exposes `user`, `session`, `userRole`, `subscriptionTier`, `subscriptionExpiresAt` and `signIn/signUp/signOut`
- New users default to the `student` role via a DB `handle_new_user` trigger

ROLES & SUBSCRIPTION

- Roles read from `user_roles`; effective role by priority `admin` › `teacher` › `student`
- Subscription tier/expiry read from `profiles`; premium checks enforced in Edge Functions, not just the client
- Routes protected by `ProtectedRoute` (allowed-roles); data protected by Postgres `RLS` + `has_role()`

10 Security Posture & Extension Points

STRENGTHS

- All AI / third-party secrets held server-side in Edge Functions; never shipped to the browser
- Chatbot input validated (count, length, roles) before reaching the model
- Premium access re-checked server-side; privileged writes use the service role only after authorization
- Row-Level Security + `has_role()` as the data-access backbone

OPPORTUNITIES (ROADMAP)

- **Context-aware roadmaps** — inject the student's enrolled courses & progress into the chatbot system prompt
- **Persist roadmaps** — store generated study plans (e.g. a `roadmaps` table) for resume/track
- Rate-limit `ai-chat` per user; verify JWT signature (not just decode) at the edge
- Wire chat replies to render markdown; add conversation history persistence

11 Module File Inventory (the extracted learning module)

Learnity — Learning Module & AI Roadmap Chatbot · Architecture

Generated 2026-07-27 · from source at commit b88c27f

The concrete source files that constitute the learning module & chatbot, for lift-and-shift or focused review.

CHATBOT

Client	src/components/AIAssistant.tsx
Backend	supabase/functions/ai-chat/index.ts
Mounted in	src/App.tsx

COURSE DELIVERY

Pages	pages/Courses.tsx · CourseDetail.tsx
Viewer	pages/dashboard/student/CourseViewer.tsx
My courses	pages/dashboard/student/MyCourses.tsx
Materials	components/dashboard/MaterialsList.tsx
Enroll fn	functions/create-enrollment

ASSESSMENT & PROCTORING

List	components/.../student/AssignmentsList.tsx
Test	components/.../student/TakeTest.tsx
Submit	components/.../student/SubmitAssignment.tsx
Proctoring	src/hooks/useProctoring.ts

PROGRESS · CERTIFICATES · TRANSCRIPTS

Progress	components/dashboard/ProgressTracker.tsx
Enrolled	components/dashboard/EnrolledCoursesList.tsx
Certificates	components/dashboard/CertificateManager.tsx
Transcript UI	components/dashboard/TranscriptViewer.tsx
Transcribe fn	functions/transcribe-video

PLATFORM / CROSS-CUTTING

Auth	src/hooks/useAuth.tsx
Guards	components/ProtectedRoute.tsx
Layout / nav	components/layout/DashboardLayout.tsx
Supabase client	src/integrations/supabase/client.ts
DB types	src/integrations/supabase/types.ts

DATABASE

Migrations	supabase/migrations/*.sql (36 files)
Core tables	courses, enrollments, modules, lessons, course_resources, assignments, assignment_submissions, certificates, transcripts, proctoring_*

Summary. The learning module is a cleanly layered slice: a React client for discovery, learning, assessment and certification; an always-on AI chatbot for customized roadmaps and guidance; and a Supabase backend where Edge Functions broker every privileged action and every AI call (Gemini 2.5 Flash for chat, Groq Whisper + Llama 3.3 for transcription), with Postgres RLS and role-based access as the security backbone.